Rapport de Synthèse Global : Project Start Trek

Architecture Logicielle, Algorithmes d'Apprentissage par Renforcement et Protocoles d'Évaluation

Partie 1 : Architecture Système et Évolution du Pipeline d'Entraînement

L'infrastructure logicielle d'un projet d'apprentissage par renforcement conditionne directement sa viabilité expérimentale. Dans le cadre de la résolution de l'environnement LunarLander-v3, le script train.py fait office de chef d'orchestre global. Sa responsabilité fondamentale couvre l'initialisation de l'environnement physique de Gymnasium, l'instanciation des agents, le pilotage des boucles d'apprentissage et la collecte systématique des variables d'état et de performance pour l'analyse ultérieure. Le développement de ce pipeline a suivi une trajectoire incrémentale structurée autour de trois phases majeures d'optimisation.

1.1 La Phase Fondatrice : L'Architecture en Dur

Au commencement des expérimentations, le script présentait une structure brute où l'ensemble des paramètres d'exécution. tels que le taux d'apprentissage (learning rate), le volume d'épisodes, la taille des lots (batch size) ou la configuration des couches masquées — étaient directement écrits au sein du code source. Si cette approche rudimentaire s'est avérée suffisante pour valider le comportement initial du module de descente, elle a rapidement montré des limites critiques lors de la transition vers une phase de recherche systématique d'hyperparamètres (Grid Search). Toute modification de stratégie ou d'architecture imposait une réécriture du code principal, augmentant le risque d'erreur humaine et annihilant toute scalabilité logicielle.

1.2 La Phase de Découplage : Le Passage aux Configurations YAML

Pour pallier la rigidité du code en dur, l'infrastructure a fait l'objet d'une refonte axée sur la modularité. L'ensemble des variables de contrôle du système a été extrait du script d'exécution pour être consigné dans des fichiers de configuration externes adoptant le format standard .yaml. Un module intermédiaire, config_loader.py, a été développé pour lire, valider et injecter dynamiquement ces configurations au sein du script train.py. Ce découplage strict a permis d'automatiser l'intégralité du processus d'expérimentation. Dès lors, le lancement de vagues d'entraînement distinctes reposait uniquement sur la spécification de fichiers de configuration cibles, préservant l'intégrité du code source principal.

1.3 La Phase de Maturité : L'Architecture Orientée Objet

L'introduction de nouveaux algorithmes (Double DQN et Proximal Policy Optimization) a nécessité une transition vers une modélisation orientée objet afin d'éviter la prolifération de structures conditionnelles complexes au sein du pipeline d'entraînement. En s'inspirant de nos anciens projet epitech, une interface de base nommée BaseAgent a été modélisée. Cette classe définit un contrat d'interface auquel se conforment tous les algorithmes du projet, imposant l'implémentation des méthodes suivantes :

- select_action : Détermination de la réponse motrice de l'agent en fonction du vecteur d'état.
- step : Enregistrement de l'expérience vécue et gestion des structures de mémoire.
- update : Exécution des algorithmes d'optimisation et mise à jour des poids synaptiques.
- get_logs : Extraction des métriques internes de performance.

Cette abstraction permet à train.py d'interagir avec les fichiers d'agents spécifiques (dqn_agent.py, doubledqn_agent.py, ppo_agent.py) de manière totalement transparente.

1.4 Robustesse de l'Architecture Finale

Le script d'entraînement final intègre l'ensemble des critères d'ingénierie requis pour la validation scientifique du projet :

- **Garantie de Reproductibilité (One-click Repro) :** L'intégration de la routine set_global_seed(seed) verrouille l'aléa du système en fixant de manière identique les générateurs de nombres aléatoires de Python, PyTorch et de l'environnement Gymnasium.
- **Instrumentation et Visualisation :** L'utilisation du module SummaryWriter de TensorBoard permet le suivi en temps réel des courbes de perte (Loss), d'évolution du taux d'exploration (epsilon) et de durée des épisodes. Le wrapper RecordVideo capture de façon périodique le rendu visuel du vaisseau sous forme de fichiers MP4.
- **Analyse de Fin de Partie :** Chaque épisode est audité pour classifier précisément la cause d'arrêt de la simulation (crash, sortie définitive du champ visuel, extinction des moteurs ou atterrissage stabilisé), alimentant ainsi une base de données de logs qualitatifs.

Partie 2 : Approche Off-Policy : Algorithme Deep Q-Network (DQN)

L'environnement LunarLander-v3 oppose à l'agent un espace d'états continu à huit dimensions (coordonnées de position, vecteurs de vitesse linéaire, angle d'assiette, vitesse angulaire et indicateurs de contact des suspensions). Face à l'infinité des combinaisons possibles, une table de correspondance classique s'avère inapplicable. L'algorithme DQN résout cette

problématique en exploitant un Perceptron Multicouche (MLP) comme approximateur de fonction de valeur de l'action optimale.

2.1 Les Fondations de l'Architecture DQN

Le modèle s'appuie sur des couches denses associées à des fonctions d'activation non linéaires de type ReLU pour traiter les variables physiques de l'environnement et attribuer un score de qualité à chacune des quatre actions discrètes du propulseur. La dynamique d'apprentissage s'articule autour de trois composants techniques fondamentaux :

- **Politique Epsilon-Greedy :** L'agent navigue entre l'exploration de l'environnement et l'exploitation de ses connaissances à l'aide d'un paramètre epsilon. Un taux de décroissance progressif (Epsilon Decay) réduit cette probabilité d'exploration au fil des épisodes pour stabiliser la politique gloutonne finale.
- **Experience Replay Buffer :** Pour briser la corrélation temporelle inhérente à la collecte séquentielle des données de trajectoire, les transitions vécues par l'agent sont stockées dans un tampon de mémoire à capacité finie. L'entraînement s'effectue en échantillonnant de manière uniforme des lots (mini-batches) de transitions passées, garantissant le respect des hypothèses de distribution des algorithmes d'optimisation.
- **Réseau Cible (Target Network) :** Afin de résoudre le problème de la cible mouvante (moving target), où le réseau ajuste ses poids vers une estimation qui change simultanément, l'architecture sépare le réseau local de prédiction du réseau cible utilisé pour formuler la cible de différence temporelle de Bellman.

2.2 Évolution Théorique : L'Extension Double DQN

Afin de corriger le biais de surestimation des valeurs de Q structurellement présent dans le DQN standard, l'architecture Double DQN a été développée. Cette variante découple la sélection de l'action de son évaluation en exploitant la dualité des réseaux : le réseau local détermine l'action optimale pour l'état futur, tandis que le réseau cible estime la valeur de cette action spécifique. Les expérimentations menées via notre protocole de recherche ont mis en évidence une convergence plus complexe et chaotique pour cette architecture spécifique sur cet environnement, confirmant la nécessité de se concentrer sur l'optimisation fine des hyperparamètres du DQN classique.

2.3 Recherche d'Hyperparamètres et Stratégie d'Ablation

Une recherche systématique (Grid Search) a été menée sur les axes critiques du DQN pour identifier les facteurs d'instabilité et neutraliser le phénomène de surapprentissage (overfitting) :

- **Sensibilité à la Taille du Réseau (Hidden Size) :** L'analyse comparative entre différentes densités de neurones a démontré que les architectures trop lourdes dégradent la capacité de généralisation spatiale de l'agent. Le réseau mémorise les spécificités du buffer d'expérience au détriment de la modélisation de la physique globale du vol.

- **Dynamique de Mise à Jour (Update Type) :** La confrontation entre le mode de rafraîchissement abrupt (Hard Update) et le mode de lissage continu (Soft Update, régi par le paramètre tau = 0.001) a validé la supériorité de la mise à jour douce, cette dernière évitant les ruptures brutales de politique de pilotage.
- **Contrôle du Taux d'Apprentissage (Learning Rate) et de la Mémoire :** Un ajustement précis de la vitesse de convergence a permis d'éliminer les comportements oscillatoires destructeurs, tandis que le calibrage de la taille du buffer a empêché l'accumulation de trajectoires obsolètes issues des phases initiales d'exploration grossière.

Partie 3 : Approche On-Policy : Proximal Policy Optimization (PPO)

Pour diversifier notre cadre d'analyse face aux méthodes fondées sur l'estimation des valeurs de Q, un agent reposant sur l'algorithme Proximal Policy Optimization (PPO) a été intégré au projet. Contrairement au DQN, PPO se classe parmi les méthodes On-Policy, ce qui implique qu'il améliore directement la politique à l'origine de la collecte de données, sans recours à un historique d'expériences.

3.1 Fondements de l'Architecture Acteur-Critique

L'algorithme PPO opère à travers une modélisation duale associant deux entités distinctes et indépendantes qui traitent le vecteur d'état de la simulation :

- **Le Réseau Acteur :** En charge du processus de prise de décision, il génère une distribution de probabilité sur l'espace discret des actions disponibles.
- **Le Réseau Critique :** Faisant office d'évaluateur, il estime la fonction de valeur d'état, prédisant le score de récompense attendu à long terme depuis l'état courant.

Le mécanisme d'apprentissage repose sur le calcul de la fonction d'Avantage. L'avantage évalue si l'action sélectionnée par l'Acteur s'avère plus ou moins performante que la prédiction moyenne formulée par le Critique. Une valeur positive indique une décision pertinente à encourager, tandis qu'une valeur négative sanctionne un choix sous-optimal, incitant le Critique à ajuster ses prédictions futures pour converger vers la réalité des récompenses récoltées.

3.2 Mécanisme de Sûreté : La Fonction d'Objectif Tronquée

La spécificité mathématique de PPO réside dans sa capacité à encadrer la mise à jour des paramètres du réseau de neurones pour éviter l'effondrement de la politique. Cet encadrement est régi par une fonction d'objectif de substitution tronquée (Clipped Surrogate Objective), nommée.

Cette fonction calcule le ratio entre la probabilité de l'action sous la politique actuelle et sa probabilité sous l'ancienne politique. Ce ratio est ensuite multiplié par la valeur d'Avantage. Afin

d'interdire des modifications de poids trop radicales qui détruiraient l'apprentissage de manière irréversible, la fonction applique un opérateur de troncation (clip) basé sur un hyperparamètre Epsilon. Si le ratio de probabilité s'éloigne de l'intervalle de sécurité prédéfini, la fonction de perte est plafonnée. Le système sélectionne systématiquement la valeur minimale entre l'objectif brut et l'objectif tronqué, neutralisant ainsi toute incitation au dépassement de la zone de confiance.

3.3 Logique Temporelle et Rétropropagation

Le déploiement de PPO a imposé une adaptation structurelle de la boucle de calcul du projet. Alors que le DQN met à jour son réseau de façon continue à chaque pas de temps, PPO accumule les trajectoires et les informations de probabilités au sein d'une mémoire de lot dédiée pendant un nombre fixe d'étapes de simulation. La mise à jour globale s'effectue à la fin de cette séquence d'accumulation.

Pour forcer l'optimisation vers l'atteinte des scores les plus élevés (gradient ascendant) au sein d'outils nativement configurés pour la minimisation des erreurs (gradient descendant), l'implémentation logicielle réalise une inversion de signe sur les fonctions de perte calculées. La rétropropagation du gradient (backpropagation) parcourt ensuite les nœuds du réseau pour réajuster la valeur des poids sur les axes de connexion, en fonction du taux d'apprentissage et de l'historique récent des trajectoires de vol.

Partie 4 : Protocole d'Évaluation, Robustesse et Spécifications Logicielles

La validation d'un modèle d'apprentissage par renforcement exige un protocole d'évaluation hermétique, capable de dissocier la performance mémorisée de la capacité réelle de généralisation. Le script eval.py remplit cette fonction d'audit final pour l'ensemble des architectures développées (DQN, Double DQN, PPO).

4.1 Principes Directeurs d'Évaluation Déterministe

En phase d'évaluation, l'agent est soumis à des contraintes d'exploitation strictes :

- **Suppression de l'Aléatoire :** Le taux d'exploration est annulé (epsilon = 0.0), interdisant toute action de pur hasard.
- **Sélection Gloutonne Pure :** L'agent sélectionne ses actions de manière strictement déterministe sur les valeurs issues de son réseau de neurones profonds.
- **Graines Multiples (Seeds) :** Pour garantir la validité scientifique, le modèle est évalué sur différentes graines environnementales définies par la configuration YAML, modifiant les profils de terrain, la force des rafales de vent et la position de départ de l'atterrisseur.

4.2 Métriques d'Analyse Statistique Globale

Pour prouver la stabilité de l'apprentissage et s'assurer que les performances observées ne dépendent pas de conditions initiales favorables, le script implémente les outils mathématiques de la statistique descriptive :

- **Moyenne Échantillonnale :** Calculée sur un volume d'épisodes consécutifs par graine environnementale pour caractériser la tendance centrale des performances de l'agent.
- **Écart-Type :** Mesure indispensable de la dispersion des scores, elle quantifie la stabilité ou l'instabilité structurelle du pilotage face aux perturbations extérieures.
- **Intervalle de Confiance à 95% :** Calculé à partir de la variance et de la taille de l'échantillon, cet indicateur encadre mathématiquement la vraie performance moyenne de l'IA avec un niveau de certitude scientifique standardisé.

4.3 Fonctionnalités Avancées d'Audit et Résilience

Le script eval.py génère un rapport complet d'évaluation à travers plusieurs canaux de sortie :

- **Audit Comportemental Qualitatif :** Le script réalise une classification systématique des causes de fin de chaque épisode en isolant les cas d'atterrissage réussi, de collision fatale (crash), de dérive hors du champ visuel (out-of-view), ou d'épuisement du temps alloué (épisode tronqué). Cette répartition permet d'identifier les faiblesses géométriques ou temporelles de la politique de l'agent.
- **Sauvegarde et Preuves Visuelles :** L'intégralité des variables de performance de chaque épisode est consignée en continu dans un fichier de logs au format CSV. En parallèle, l'environnement est encapsulé par le wrapper vidéo pour générer des fichiers de preuve au format MP4.
- **Résilience du Code (Fallback Paths) :** Le script intègre un système de sécurité logiciel pour le chargement des poids des modèles. Si le chemin spécifié dans le fichier YAML est inaccessible, un mécanisme de secours redirige automatiquement la recherche vers le répertoire de sauvegarde par défaut des actifs pré-entraînés du projet, évitant ainsi l'interruption prématurée du programme d'évaluation.